LoRide-R

Custom Project Report



EE329 – Section – 01

Fall 2025

Leo Alvelais, Anthony di Furia, Juan Gonzalez, Ches Zepp

Instructor: John L. Penvenne

Date 12/5/2025

Behavior Description

LoRide's purpose is to act as a wireless, vehicle health monitoring device. The system is comprised of two nodes each outfitted with an RFM95W LoRa transceiver and a Nucleo-L4A6ZG microcontroller. The two nodes communicate wirelessly using a LoRa modulation scheme at 915MHz. The node on the vehicle is additionally outfitted with an OBD2 connector which receives the vehicle data via the CANbus communication protocol and MCP2515 controller then transmit it through SPI into the Arduino, which then the Arduino transmits the data via UART3 to the STM32. Sensor indicators and information pertaining to the vehicle's health such as air intake temperature, coolant temperature, and speed are gathered, encoded into a data stream of 10 bytes, and transmitted wirelessly over the air to the 'base' node. Sensor indicators and information received at the base node are printed to a terminal emulator over a UART communication port.

System Specification

Table 1: Total System parameters for Base station

Bandwidth [kHz]	125
Modulation Scheme	LoRa
Carrier Frequency [MHz]	915
Transmit Power [dBm]	15
Max Current Draw [mA]	100.5
Minimum Current Draw [mA]	2.3
Max Receiver Sensitivity [dBm]	-116
Nucleo Clock Frequency [MHz]	2
Microcontroller	Nucleo-L4A6zG
Supply Voltage [V]	5
Over the Air Transmission Time [ms]	250
UART baud rate	115200

Table 2: Total System parameters for Vehicle Node

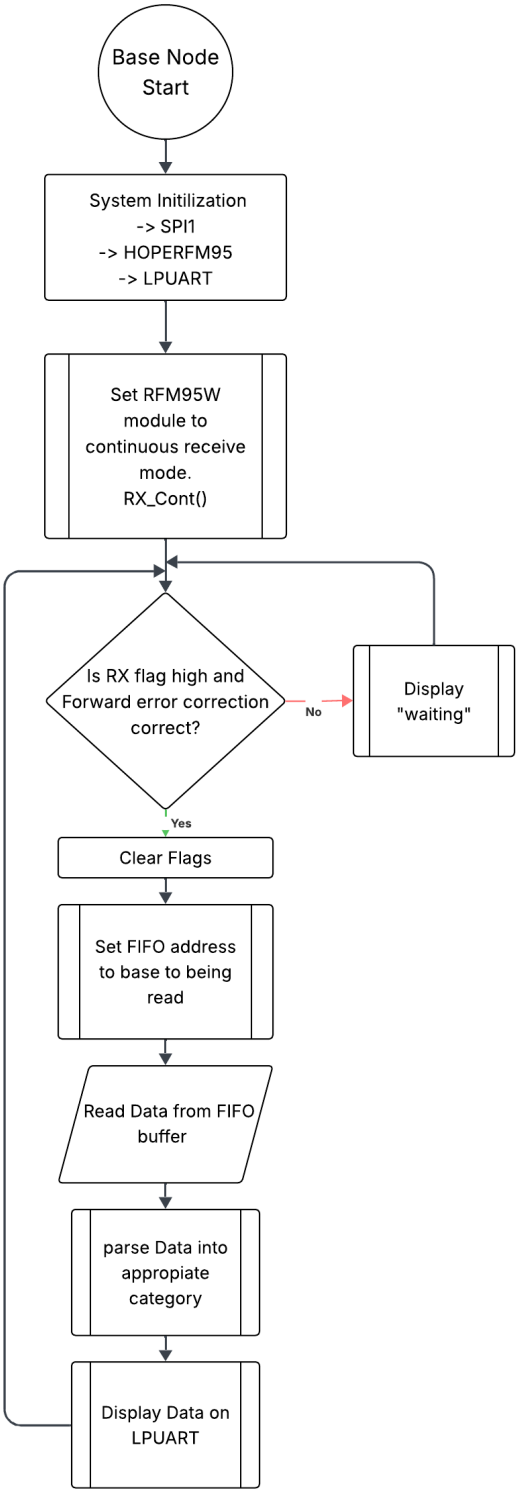
Modulation Scheme	LoRa
Bandwidth[kHz]	125
Carrier Frequency [MHz]	915
Transmit Power [dBm]	14
Max Current Draw [mA]	100.5
Minimum Current Draw [mA]	2.3
Max Receiver Sensitivity [dBm]	-116
Nucleo Clock Frequency [MHz]	2
Microcontroller	Nucleo-L4A6zG
Supply Voltage [V]	5
Over the air Transmisison time [ms]	250
UART baud rate	115200
Max Current Draw [mA]	250
Minimum Current Draw [mA]	125
OBD2 Interface	Seed Studio CANbus shield
OBD2 Microcontroller	Arduino
OBD2 speed	500 kbps
OBD2 max Parameter Identifiers [PID] Poling	50
OBD2 Protocol	ISO 15765-4

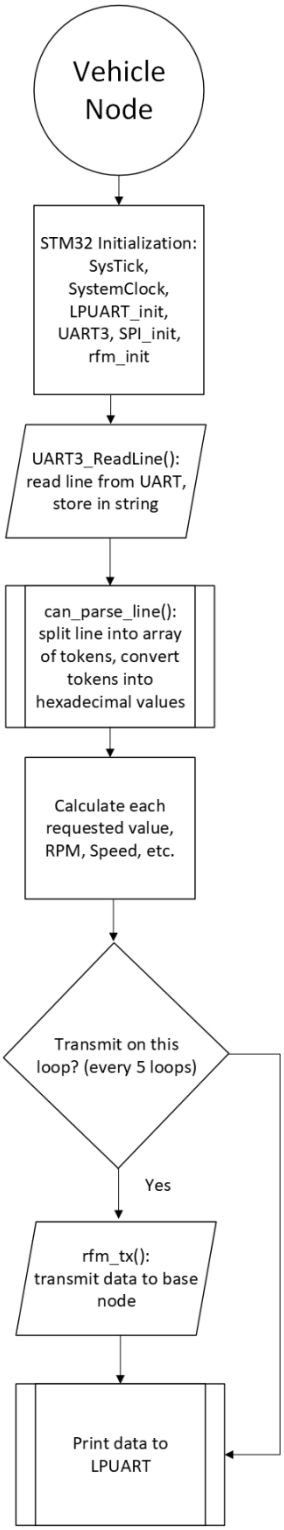
Code Planning

This code implements a one-way wireless telemetry link between a beacon vehicle node and a base station node between two NUCLEO-L4A6ZG microcontrollers equipped with RFM95W LoRa transceiver modules. The vehicle node periodically samples OBD-II sensor data from the CAN bus, fixes each sensor indicator into fixed 2-byte frames, and streams packets over LoRa with no acknowledgement from the base station or “downlink”. This sampling is achieved by using a CAN bus shield attached to an Arduino, which communicates with the vehicle node’s STM32 via UART3 The base-station node is continuously receiving, polling for valid packets. When a valid packet is received, the 2-byte sensor fields are unpacked, and displayed via UART onto a terminal emulator for user optics.

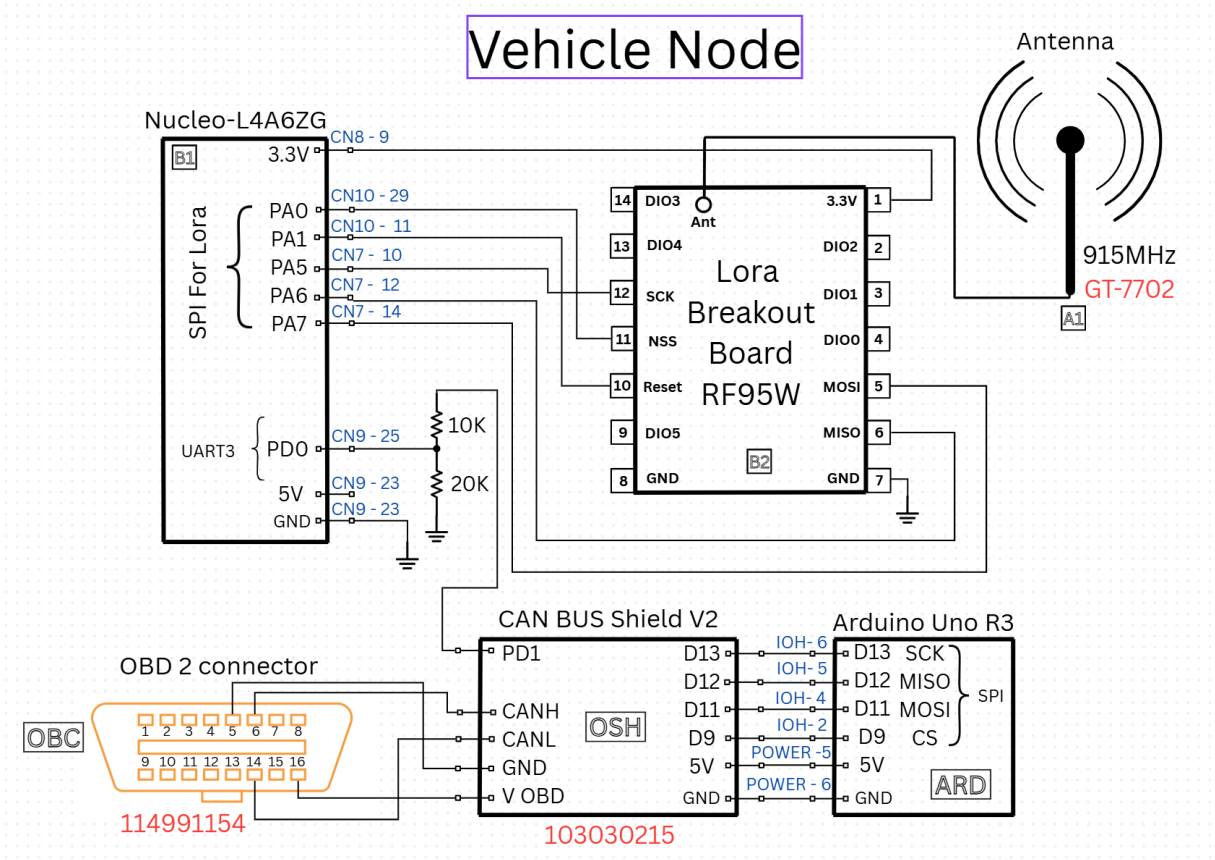
The vehicle node receives ASCII-formatted CAN frames, 32 bit strings, via UART3 from an Arduino connected CAN shield, which arrives containing the CAN PID, data length, and payload bytes. Once the raw binary data is available, decoder functions validate the OBD-II response format by verifying the correct mode and parameter identification codes before applying conversion formulas to extract meaningful vehicle parameters such as engine RPM, wheel speed, coolant temperature, mass airflow, and intake air temperature. The data obtained is known to be in the metric system in which the raw sensor values are then converted into practical units like miles per hour and degrees Fahrenheit. After extraction, the vehicle parameters are consolidated into a compact binary data stream where each metric is represented as a two-byte value split into high and low bytes, creating a space-efficient ten-byte payload directly via SPI to the LoRa transmission system for wireless communication to a remote base station.

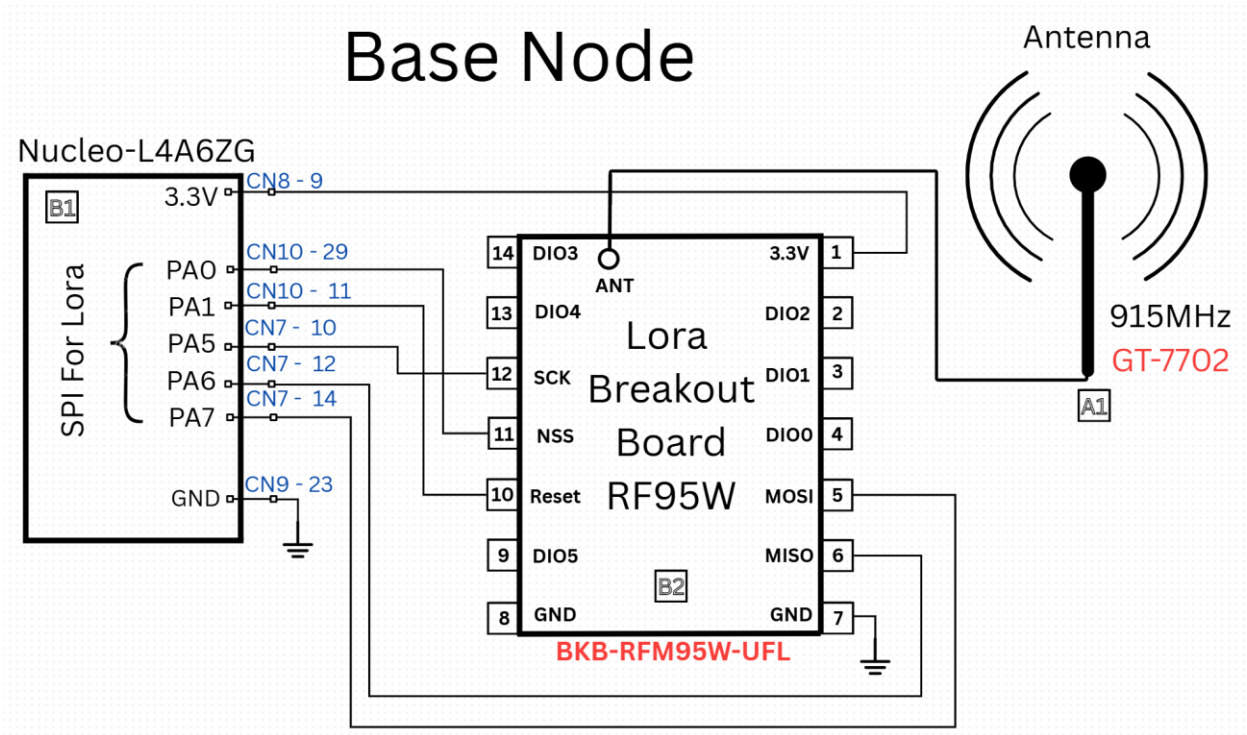
Flowcharts


Base Node Start
System Initilization -> SPI1 -> HOPERFM95 -> LPUART
Set RFM95W module to continuous receive mode. RX_Cont()
Is RX flag high and Forward error correction correct?
No
Display "waiting"
Yes
Clear Flags
Set FIFO address to base to being read
Read Data from FIFO buffer
parse Data into appropiate category
Display Data on LPUART


Vehicle Node
STM32 Initialization: SysTick, SystemClock, LPUART_init, UART3, SPI_init, rfm_init
UART3_ReadLine(): read line from UART, store in string
can_parse_line(): split line into array of tokens, convert tokens into hexadecimal values
Calculate each requested value, RPM, Speed, etc.
Transmit on this loop? (every 5 loops)
Yes
rfm_tx(): transmit data to base node
Print data to LPUART

System Schematic





by Juan Gonzalez

Attribution
***references to their source material in reference sections**

TheGreyDiamond

Ultimately TheGreyDiamond's Arduino experimental code did not work for us. But he did give us the great idea to utilize our already owned SDR# antenna and software to be able to monitor our packet sends over the air without the need of a working receive module. This was in invaluable testing tool we used throughout development to confirm transmission success.

Nikita91 (Lead II) *STMicroelectronics Community*

After Alvelais had structured the SPI protocol to send 8bit packets to the LoRa module per the sx127 datasheet, it became apparent on the logic analyzer that the STM32 Nucleo still sent SPI packets of size 16. After attempting to adjust different SPI settings, a visit to the STM32 products forum page yielded a user addressing this exact problem. Nikita91 suggested his line of code

((__IO uint8_t *) & pSpi->DR) = (uint8_t) data [i] ;

Which was adjusted for our case and yeilds in the code as,

*(__IO uint8_t *)&SPI1->DR = b;

CSS electronics

They provide an overview of can bus protocol as well as the different specifications that are implemented in vehicles. This helped us understand the various can data configurations that vehicle manufacturers use to output data, thankfully since vehicle manfucaturers have to follow standard obd2 protocol, the vehicle sensors and ID's are defined by SAE International Organization for all vehicles. If not we would have had to figure out each pid which who knows how long it would have taken us to get to work.

SEEED Studio

The provided libraries from Seeed Studio were helpful in the implementation of the CAN BUS aspect of the project. Some simple modifications were made to the provided code to request certain PID's from the vehicle, which returned to the Arduino's serial monitor. They also make the CAN BUS breakout board and OBD2 connector wire that was used to pull data from the vehicle computer.

Appendix

A. Bill of Materials

EE329 Custom Project – LoRide				
qty	part type	designator/id	description	Vendor: p/n
2	MCU	B1	STM32-L4A6ZG	DigiKey: 497-NUCLEO-L4A6ZG-ND
2	Radio Transceiver	B2	HopeRF RFM95W LoRa 915 MHz Transceiver (SX1276-based)	Amazon ASIN: B0BKYJG7T4 (Seller: Lubbey Smart)
2	Antenna	A1	915MHz Antenna	Amazon ASIN: B0FJLVVQZB (Seller: Meshnology Store) Vendor Part Number: GT-7702
2	Antenna Cables	C1	20cm SMA male U.FL female	Amazon ASIN: B0FJLVVQZB (Seller: Meshnology Store) Vendor Part Number: GT-7702
2	Antenna cable connection to PCB	UF1	Surface Mount connection bridging antenna to PCB	DQLZV Official Store (AliExpress) 20279-001E
1	Seeed studio OBD2 connector	OBC	Connects OBD2 from car to shield	Vendor: Seeed Studio 114991154
1	Seeed studio CAN BUS Shield V2	OSH	Connects OBD2 transceiver to Arduino	Vendor: Seeed Studio 103030215
1	Ardunio Uno	ARD	microcontroller	Arduino Store A000066

B. References

[1] Arduino, UNO R3 — Technical Documentation. [Online]. Available: https://docs.arduino.cc/hardware/uno-r3

[2] HopeRF Electronic, RFM95/96/97/98(W) – Low Power Long Range Transceiver Module, Ver. 1.0. [Online]. Available: https://www.hoperf.com/data/upload/portal/20190801/RFM95_96_97_98(W).pdf

[3] Nikita91, "SPI configured to send 8-bit sends 16-bits," STMicroelectronics Community Forum. [Online]. Available: https://community.st.com/t5/stm32-mcus-products/spi-configured-to-send-8bit-sends-16bits/td-p/163387

[4] Seeed Studio, CAN-BUS Shield V2.0 — Hardware Overview and Documentation. [Online]. Available: https://wiki.seeedstudio.com/CAN-BUS_Shield_V2.0/

[5] STMicroelectronics, RM0351: STM32L47xx, STM32L48xx, STM32L49xx and STM32L4Axx Advanced Arm®-Based 32-Bit MCUs Reference Manual. [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0351-stm32l47xxx-stm32l48xxx-stm32l49xxx-and-stm32l4axxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

[6] STMicroelectronics, STM32L4A6xG: Ultra-Low-Power Arm® Cortex®-M4 MCU Datasheet, Production Data. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32l4a6vg.pdf

[7] TheGreyDiamond, "Getting Started with RFM95W, LoRa and ESP32," TheGreyDiamond Blog, Sep. 21, 2021. [Online]. Available: https://thegreydiamond.de/blog/2021/09/21/getting-started-with-rfm95w-lora-and-esp32/

[8] CSS Electronics, "OBD2 Explained - A Simple Intro [2025]," [Online]. Available: https://www.csselectronics.com/pages/obd2-explained-simple-intro

C. Source Code

1. Base Station

---

main.h

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.h
  * @brief          : Header for main.c file.
  *                   This file contains the common defines of the application.
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */

/* Define to prevent recursive inclusion -------------------------------------*/
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

/* Includes ------------------------------------------------------------------*/
#include "stm32l4xx_hal.h"



void Error_Handler(void);



#define B1_Pin GPIO_PIN_13
#define B1_GPIO_Port GPIOC
#define LD3_Pin GPIO_PIN_14
#define LD3_GPIO_Port GPIOB
#define USB_OverCurrent_Pin GPIO_PIN_5
#define USB_OverCurrent_GPIO_Port GPIOG
```

```
#define USB_PowerSwitchOn_Pin GPIO_PIN_6
#define USB_PowerSwitchOn_GPIO_Port GPIOG
#define STLK_RX_Pin GPIO_PIN_7
#define STLK_RX_GPIO_Port GPIOG
#define STLK_TX_Pin GPIO_PIN_8
#define STLK_TX_GPIO_Port GPIOG
#define USB_SOF_Pin GPIO_PIN_8
#define USB_SOF_GPIO_Port GPIOA
#define USB_VBUS_Pin GPIO_PIN_9
#define USB_VBUS_GPIO_Port GPIOA
#define USB_ID_Pin GPIO_PIN_10
#define USB_ID_GPIO_Port GPIOA
#define USB_DM_Pin GPIO_PIN_11
#define USB_DM_GPIO_Port GPIOA
#define USB_DP_Pin GPIO_PIN_12
#define USB_DP_GPIO_Port GPIOA
#define TMS_Pin GPIO_PIN_13
#define TMS_GPIO_Port GPIOA
#define TCK_Pin GPIO_PIN_14
#define TCK_GPIO_Port GPIOA
#define SWO_Pin GPIO_PIN_3
#define SWO_GPIO_Port GPIOB
#define LD2_Pin GPIO_PIN_7
#define LD2_GPIO_Port GPIOB

/* USER CODE BEGIN Private defines */

/* USER CODE END Private defines */

#ifdef __cplusplus
}
#endif

#endif /* __MAIN_H */
```

---

main.c

```
/******************************************************************************
 * EE 329 CUSTOM PROJECT
 ******************************************************************************
 * @file           : main.c
 * @brief          : Enables telemetry for vehicle health monitoring.
 *                   Will allow for continuous tx(beacon mode) and rx.
 * project         : EE 329 F'25 CP Assignment: LoRide-R
 * authors         : Leo Alvelais, Anthony DeFuria, Ches Zepp, Juan Gonzalez.
 * version         : 0.5
 * date            : 25125
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2023 STMicroelectronics.  All rights reserved.
 ******************************************************************************
 * CP Plan :
 * Enable SPI1 communication between RFM95W transceiver module and NUCLEO.
 * Enable OBD-II communication between CANbus shield and vehicle CAN bus.
 * Enable UART communication between Arduino and NUCLEO.
 * Configure the operation mode and modem for RFM95W module.
 * Create functions to demonstrate simple wireless pitch and catch via LoRa.
 * Create modular functions for the vehicle and base station nodes
 * Process packet payloads into sensor indicators/quantities for display.
 ******************************************************************************
 * REVISION HISTORY
 * 0.1: Configured communication via SPI between the nucleo and rfm95,
 * programmed the op mode and modem of the rfm95, add function for tx and rx.
 * 0.2: Enabled communication via UART between CAN bus shield and nucleo.
 * 0.3: Demonstrated pitch and catch over LoRa of 32 byte packet.
 * 0.4: Demonstrated wireless transmission and reception of sensor information
 * and printed sensor information to terminal emulator.
 * ***********************
 * TODO
 * Add extra features:
 * Wireless command of car functions.
 * Enable faster communication by reconfiguring the system clock.
 ******************************************************************************


 * 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567 */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "SPI.h"
#include "LPUART.h"
#include "delay.h"
#include "UART3.h"
#include "can_read.h"
```

```
void SystemClock_Config(void);



int main(void)
{

   //INITIALIZE ALL PERIPHERALS, CLOCKS AND COM PORTS
  HAL_Init();

  SystemClock_Config();




  SysTick_Init();
  SPI_Config();
  SPI_init();

  rfm_init();
  LPUART1_init();

  //USE EITHER Vehicle_node() or base_node() depending on if the NUCLEO is
  //bound to the car or acting as the base station respectively.

  //base_node();
  //Vehicle_node();



}



void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */

  if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the RCC Oscillators according to the
   * specified parameters in the RCC_OscInitTypeDef
```

```
   * structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSI;
  RCC_OscInitStruct.MSIState = RCC_MSI_ON;
  RCC_OscInitStruct.MSICalibrationValue = 0;
  RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }
  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_MSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;


  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report
   * the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

SPI.h

```
/*
 * SPI.h
 *
 *  Created on: Nov 15, 2025
 *      Author: alvel
 */


#ifndef INC_SPI_H_
#define INC_SPI_H_

//MY INCLUSIONS
#include "main.h"

//MY DEFINITIONS
#define SPI_PORT (GPIOA)
#define SPI_CP_DS (0x7 << 8) //8 bit data size
#define CS_PIN (GPIO_PIN_0)
#define RS_PIN (GPIO_PIN_1)
#define TX_DONE (1U << 3)
#define RX_DONE (1U << 6)
#define CRC_ERR (1U << 5)
#define PCK_LEN (32)

#endif /* INC_SPI_H_ */




//MY PROTOTYPES
void SPI_Config(void);
void SPI_init( void );
uint8_t SPI_wr_b(uint8_t b);
void CS_low(void);
void CS_high(void);
void RS_low(void);
void RS_high(void);
void SPI_w_nB(uint8_t *data, uint32_t n);
void SPI_rd_nB(uint8_t *data, uint32_t n);
void rfm_wr(uint8_t addr, uint8_t b);
uint8_t rfm_rd(uint8_t addr);
void rfm_wr_nB(uint8_t addr, uint8_t *Wdata, uint32_t len);
void rfm_rd_nB(uint8_t addr, uint8_t *Rdata, uint32_t len);
void rfm_init(void);
void rfm_tx(uint8_t *data, uint32_t size );
void RX_Cont(void);
uint8_t RX_polling(uint8_t *data, uint8_t max_len);
void Pitch_Demo(void);
void Catch_Demo(void);
```

```
uint8_t  RFM95_Basic_Bringup(void);
void base_node (void);
```

SPI.c

```
/*
 ******************************************************************************
 * EE 329 CUSTOM PROJECT
 ******************************************************************************
 * @file           : SPI.c
 * @Created on     : November 15, 2025
 * @brief          : Enables SPI communication, programs RFM95,
 *                   - develops functions for vehicle/base station node.
 * authors         : Leo Alvelais, Anthony DeFuria, Ches Zepp, Juan Gonzalez.
 ******************************************************************************
 *SPI Wiring plan:
 *Peripheral          ---    NUCLEO
 *POWER(3.3V)-Pin1    ---    3.3V = CN8-9
 *CS-Pin11            ---    PA0=CN10-29
 *RESET-Pin10         ---    PA1=CN10-11
 *SCK-Pin12           ---    PA5=CN7-10
 *MISO-Pin6           ---    PA6=CN7-12
 *MOSI-Pin5           ---    PA7=CN7-14
 ******************************************************************************
 */
#include "SPI.h"
#include "main.h"
#include "delay.h"
#include "LPUART.h"
#include "UART3.h"
#include "can_read.h"

/*
 * Function: SPI_Config
 * ----------------------------
 *   Configures SPI1 GPIOA ports for to configure hardware for
 *   -communication between the NUCLEO and the Hope RFM95W module.
 *   NOTE: Uses manual chip select for multi-byte write.
 */

void SPI_Config(void) {

   // enable clock for GPIOA & SPI1
   RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOAEN);              // GPIOA: DAC NSS/SCK/SDO

   RCC->APB2ENR |= (RCC_APB2ENR_SPI1EN);                 // SPI1 port

   //CONFIGURE AF FOR EACH GPIOA PORT:
   //MOSI
   GPIOA->AFR[0] &= ~((0x000F << GPIO_AFRL_AFSEL7_Pos)); // clear nibble for bit 7 AF
```

```
    GPIOA->AFR[0] |= ((0x0005 << GPIO_AFRL_AFSEL7_Pos)); // set b7 AF to SPI1 (fcn 5)

    //SCK
    GPIOA->AFR[0] &= ~((0x000F << GPIO_AFRL_AFSEL5_Pos)); // clear nibble for bit 5 AF

    GPIOA->AFR[0] |= ((0x0005 << GPIO_AFRL_AFSEL5_Pos)); // set b5 AF to SPI1 (fcn 5)

    //MISO -- PA6
    GPIOA->AFR[0] &= ~((0x000F << GPIO_AFRL_AFSEL6_Pos));

    GPIOA->AFR[0] |= ((0x0005 << GPIO_AFRL_AFSEL6_Pos));

    /* USER ADD GPIO configuration of MODER/PUPDR/OTYPER/OSPEEDR registers HERE */
    // configure AFR for SPI1 function (1 of 3 SPI bits shown here)
    //CS    -> PA0
    //RESET -> PA1
    SPI_PORT->MODER &= ~( GPIO_MODER_MODE6 | GPIO_MODER_MODE5 | GPIO_MODER_MODE7
          | GPIO_MODER_MODE0 | GPIO_MODER_MODE1);

    //AFR MODE FOR PIN 6,5 AND 7
    SPI_PORT->MODER |= ( GPIO_MODER_MODE6_1 | GPIO_MODER_MODE5_1
          | GPIO_MODER_MODE7_1);

    //PULL DOWN SCK
    SPI_PORT->PUPDR &= ~(GPIO_PUPDR_PUPD5);

    SPI_PORT->PUPDR |= GPIO_PUPDR_PUPD5_0;

    //CONFIGURE MANUAL GPIO OUTPUT FOR CS AND RESET AT HIGH SPD
    SPI_PORT->MODER |= (GPIO_MODER_MODE0_0 | GPIO_MODER_MODE1_0);

    SPI_PORT->OTYPER &= ~(GPIO_OTYPER_OT0 | GPIO_OTYPER_OT1); //PUSH PULL

    SPI_PORT->PUPDR &= ~(GPIO_PUPDR_PUPD0 | GPIO_PUPDR_PUPD1); //NO PU/PD

    //Clearing bits of the speed register prior to setting due to preallocation issues.
    SPI_PORT->OSPEEDR &= ~((3 << GPIO_OSPEEDR_OSPEED0_Pos)
          | (3 << GPIO_OSPEEDR_OSPEED1_Pos));

    SPI_PORT->OSPEEDR |= ((3 << GPIO_OSPEEDR_OSPEED0_Pos)
          | (3 << GPIO_OSPEEDR_OSPEED1_Pos)); //HIGH SPD

}

/*
 * Function: SPI_init
 * --------------------
 * Configures SPI1 for full duplex, 8-bit communication
 * build control registers CR1 & CR2 for SPI control of peripheral DAC
```

```
 */

void SPI_init(void) {

   // SPI config as specified @ STM32L4 RM0351 rev.9 p.1459

   // CR1 (reset value = 0x0000)

   SPI1->CR1 &= ~( SPI_CR1_SPE);               // disable SPI for config

   SPI1->CR1 &= ~( SPI_CR1_RXONLY);            // recv-only OFF

   SPI1->CR1 &= ~( SPI_CR1_LSBFIRST);          // data bit order MSb:LSb

   SPI1->CR1 &= ~( SPI_CR1_CPOL | SPI_CR1_CPHA); // SCLK polarity:phase = 0:0

   SPI1->CR1 |= SPI_CR1_MSTR;                // MCU is SPI controller

   SPI1->CR1 &= ~(SPI_CR1_BR_Msk);

   SPI1->CR1 |= SPI_CR1_BR_1;

   // CR2 (reset value = 0x0700 : 8b data)
   SPI1->CR2 &= ~( SPI_CR2_TXEIE | SPI_CR2_RXNEIE); // disable FIFO intrpts

   SPI1->CR2 &= ~( SPI_CR2_FRF);                // Moto frame format

   //CS PULSE IS MANUALLY GENERATED
   SPI1->CR2 |= SPI_CP_DS;                  // 8-bit data

   SPI1->CR2 |= SPI_CR2_SSOE;                // enable SS output

   // ENABLE SPI
   SPI1->CR1 |= SPI_CR1_SPE;                 // re-enable SPI for ops

   SPI1->CR2 |= SPI_CR2_FRXTH;
   //SET RS AND CS IDLE HIGH
   CS_high();

   RS_high();
}

/*
 * Function: CS_low
 * --------------------
 * Pulls the chip select ported to the RFM95 low.
 * Should be used at the start of every SPI write or read.
 */
```

```
void CS_low(void) {
   SPI_PORT->BRR |= CS_PIN;
}


/*
 * Function: CS_high
 * --------------------
 * Pulls the chip select ported to the RFM95 high.
 * Should be used at the end of every SPI write or read.
 */
void CS_high(void) {
   SPI_PORT->BSRR |= CS_PIN;
}


/*
 * Function: RS_high
 * --------------------
 * Pulls the chip select ported to the RFM95 high.
 * Should be used to set RFM95 into default settings.
 */

void RS_high(void) {
   SPI_PORT->BSRR |= RS_PIN;
}


/*
 * Function: RS_high
 * --------------------
 * Pulls the chip select ported to the RFM95 low.
 * Should be used after RS_low after a delay
 */
void RS_low(void) {
   SPI_PORT->BRR |= RS_PIN;
}


/*
 * Function:  SPI_wr_b
 * --------------------
 * Writes and receives a byte over SPI
 *
 *  n: The byte to be written.
 *
 *  returns: A byte sent back by the peripheral
 *
 *  Assumes the data bus line is never busy.
 */

uint8_t SPI_wr_b(uint8_t b) {

   while (!(SPI1->SR & SPI_SR_TXE)) {
   };
```

```
   *(__IO uint8_t*) &SPI1->DR = b;

   while (!(SPI1->SR & SPI_SR_RXNE)) {
   };

   uint8_t br = *(__IO uint8_t*) &SPI1->DR;

   return br;

}

/*
 * Function:  SPI_wr_nB
 * --------------------
 * Writes n number of bytes via SPI
 *
 *  data: bytes to be written
 *
 *  n: number of bytes to be written
 *
 *  Assumes the data bus line is never busy.
 */

void SPI_wr_nB(uint8_t *data, uint32_t n) {

   for (uint32_t W_Idx = 0; W_Idx < n; W_Idx++) {

      (void) SPI_wr_b(data[W_Idx]);
   }
}

/*
 * Function:  SPI_rd_nB
 * --------------------
 * Reads n number of bytes via SPI.
 *
 *  data: bytes to be written
 *
 *  n: number of bytes to be written
 *
 *  Assumes the data bus line is never busy.
 */
void SPI_rd_nB(uint8_t *data, uint32_t n) {

   for (uint32_t Rd_Idx = 0; Rd_Idx < n; Rd_Idx++) {

      data[Rd_Idx] = SPI_wr_b(0x00);
   }
}
```

```
/*
 * Function:  rfm_wr
 * --------------------
 * Writes a byte to rfm memory address
 *
 *  addr: address to be written to
 *
 *  b: byte to write to memory address
 *
 *  CS must be toggled low and high before and after each
 *  byte respectively.
 */
void rfm_wr(uint8_t addr, uint8_t b) {

   uint8_t frame[2] = { (0x80 | addr), b };

   CS_low();

   SPI_wr_nB(frame, 2);

   CS_high();
}

/*
 * Function:  rfm_rd
 * --------------------
 *  reads a byte from rfm memory address
 *
 * addr: address to be read
 *
 *  returns: a byte from rfm memory at specified address
 *
 *  for a single read, two byte transfers must occur,
 *  after which CS is toggled
 *  A 'dummy' byte must be sent to keep SCK high for
 *  peripheral data read.
 */

uint8_t rfm_rd(uint8_t addr) {

   CS_low();

   (void) SPI_wr_b((0x7F & addr));

   uint8_t out = SPI_wr_b(0x00);

   CS_high();

   return out;
}
```

```
/*
 * Function:  rfm_wr_nB
 * --------------------
 * Writes n number of bytes to rfm memory, only used to
 * write to the fifo buffer for this particular project.
 * Fifo buffer contains contents from received packet or
 * contents to transmit.
 *
 * addr: address to write to.
 *
 *  Wdata: Data buffer to write.
 *
 *
 *
 * CS must be toggled on and off before and after the entire
 * n-byte transfer respectively
 */

void rfm_wr_nB(uint8_t addr, uint8_t *Wdata, uint32_t len) {

   CS_low();

   SPI_wr_b((0x80 | addr));

   SPI_wr_nB(Wdata, len);

   CS_high();
}

/*
 * Function:  rfm_rd_nB
 * --------------------
 * Writes a byte to rfm memory address
 *
 * addr: address to be read from. Address increments each byte
 * transfer from memory.
 *
 * len: number of bytes to be read.
 *
 * Rdata: Data buffer for bytes to be stored into.
 */

void rfm_rd_nB(uint8_t addr, uint8_t *Rdata, uint32_t len) {

   CS_low();

   SPI_wr_b((0x7F & addr));

   SPI_rd_nB(Rdata, len);
```

```
   CS_high();
}

/*
 * Function:  Rst_pulse
 * --------------------
 * Pulses Rst pin to set rfm module into sleep mode and
 * set default settings.
 *
 * Per the rfm manual, a 100us delay must be set
 * after pulling rst low,and a 5ms delay must be
 * set after pulling rst high for proper rfm
 * functionality.
 */

void Rst_pulse(void) {

   RS_low();

   delay_us(200);

   RS_high();

   delay_ms(6);
}

/*
 * Function:  rfm_fifo_wr
 * --------------------
 * Writes n number of bytes to the fifo buffer of the rfm.
 *
 * data: Data to be written into fifo buffer.
 *
 * len: Number of bytes to be written.
 *
 * This function is to be used for transmitting data wirelessly.
 * The fifo buffer is partioned for rx and tx data, this can be changed
 * such that the entire fifo buffer is used for either rx or tx
 */
void rfm_fifo_wr(uint8_t *data, uint32_t len) {
   rfm_wr_nB(0x00, data, len);
}

/*
 * Function:  rfm_fifo_rd
 * --------------------
 * Read n number of bytes from fifo buffer
 *
 * data: Data to be written into fifo buffer.
 *
```

```
 * len: Number of bytes to be written.
 *
 */

void rfm_fifo_rd(uint8_t *data, uint8_t len) {
   rfm_rd_nB(0x00, data, len);
}


/*
 * Function:  rfm_init
 * --------------------
 * Configures the modem and operation mode of the rfm module.
 * This includes, amplifiers, modulation scheme,
 * bandwidth, carrier frequency, preamble length, spreading factor,
 * and sync word.
 *
 *
 */

void rfm_init(void) {

   Rst_pulse();
   //------OP MODE ------//
   //LONG RANGE MODE W/SLEEP
   rfm_wr(0x01, 0x80);
   delay_ms(1);
   //PUT INTO STANDBY MODE
   //HF
   //LORA REGs
   //
   rfm_wr(0x01, 0x81);

   //------CARRIER FREQUENCY------//
   //915 MHz CARRIER FREQUENCY WITH ~60 Hz STEPS
   rfm_wr(0x06, 0xE4);
   rfm_wr(0x07, 0xC0);
   rfm_wr(0x08, 0x00);

   //------PA, OSC, AND LNAg CONFIGURATIONS------//
   //+15dBm THROUGH PA BOOST
   rfm_wr(0x09, 0x8D);
   //MAX 100mA OVERCURRENT PROTECTION
   rfm_wr(0x0B, 0x2B);
   //LNA
   //MAXIMUM GAIN
   //LNA BOOST ON
   rfm_wr(0x0C, 0x23);

   //------MODEM CONFIG------//
   //BASE ADDRESSES FOR TX AND RX FIFO R FINE AS DEFAULT

   //125 kHz BANDWIDTH
```

```
   //4/6 CODING RATE
   //EXPLICIT HEADER MODE
   rfm_wr(0x1D, 0x72);
   // SF OF 7
   //CRC ON

   //rfm_wr(0x1E, 0x74);
   rfm_wr(0x1E, 0x94);

   //PREAMBLE LENGTH -- 8 SYMBOLS
   rfm_wr(0x20, 0x00);
   rfm_wr(0x21, 0x08);

   //SYNC WORD FOR NETWORK
   rfm_wr(0x39, 0x12);

   //------DIO INTERRUPTS-------//
   //ENABLE INTERRUPTS TO PROPAGATE THROUGH DIO PINS
   rfm_wr(0x11, 0x00);
   //CLEAR PENDING FLAGS8
   rfm_wr(0x12, 0xFF);

}


/*
 * Function:  rfm_tx
 * --------------------
 * Transmits n number of bytes.
 *
 * data: Data to be transmitted.
 *
 * size: Size of packet to be transmitted
 *
 */

void rfm_tx(uint8_t *data, uint32_t size) {
   //SET FIFO BUFFER STARTING ADDRESS TO 0X80
   rfm_wr(0x0D, 0x80);

   //SPECIFY THE SIZE OF THE PAYLOAD TO BE TX
   rfm_wr(0x22, size);

   //WRITE DATA TO FIFO BUFFER
   rfm_fifo_wr(data, size);

   //CLEAR PENDING INTERRUPTS
   rfm_wr(0x12, 0xFF);

   //MAP TX DONE INT TO DIO0
   rfm_wr(0x40, 0x40);
```

```
   //TURN ON TX MODE IN REGOP REG
   rfm_wr(0x01, 0x83);

   //WAIT FOR TX DONE
   while (!(rfm_rd(0x12) & TX_DONE)) {
   }

   //CLEAR TX DONE INT
   rfm_wr(0x12, TX_DONE);

   //SET BACK TO STANDBY MODE
   rfm_wr(0x01, 0x81);

}

/*
 * Function: RX_Cont
 * --------------------
 * Sets the rfm into continuous receive mode
 *
 */

void RX_Cont(void) {
   //CLEAR PENDING FLAGS
   rfm_wr(0x12, 0xFF);

   //SET FIFO BASE ADDR PTR TO FIFO RX BASE
   rfm_wr(0x0D, 0x00);

   //MAP DIO0 TO RXDONE FLAG
   rfm_wr(0x40, 0x00);

   //STDBY--> CONT RX
   rfm_wr(0x01, 0x85);

}

/*
 * Function:  RX_polling
 * --------------------
 * Polls for valid packet, ie valid crc and makes sure rx complete.
 * Stores received data into buffer along.
 * Truncates packet payload if packet is longer than expected.
 *
 * data: Data buffer for received packet payload.
 *
 * max_len: The max length in bytes of the payload to be received.
 *
 */
```

```
uint8_t RX_polling(uint8_t *data, uint8_t max_len) {

   //RD FLAGS CHECK FOR CRC ERR AND RX DONE
   uint8_t flags = rfm_rd(0x12);

   if (flags & RX_DONE) {

      if (!(flags & CRC_ERR)) {

         uint8_t rx_pckt_sz = rfm_rd(0x13);

         if (rx_pckt_sz > max_len) {

            rx_pckt_sz = max_len;
         }

         //RECEIVED PACKET STARTING ADDR

         uint8_t addr = rfm_rd(0x10);

         //SET FIFO ADDR POINTER TO RC PCKT ADDR IN FIFO

         rfm_wr(0x0D, addr);

         //READ PACKET

         rfm_fifo_rd(data, rx_pckt_sz);

         //CLEAR PENDING FLAGS

         rfm_wr(0x12, (RX_DONE | CRC_ERR));

         return rx_pckt_sz;
      } else {
         rfm_wr(0x12, (RX_DONE | CRC_ERR));
      }
   }
   return 0;
}

/*
 * Function: Pitch_Demo
 * Demonstrates a 32 Byte tx over LoRa
 * --------------------


 *
 */
```

```
void Pitch_Demo(void) {

   uint8_t sz = 32;

   uint8_t msg[sz];

   while (1) {

      for (uint8_t Idx = 0; Idx < 32; Idx++) {

         msg[Idx] = Idx;

      }

      rfm_tx(msg, sz);

      delay_ms(500);
   }

}

/*
 * Function: Catch_Demo
 * --------------------
 * Demonstrates receiving a 32 byte packet over LoRa.
 */
void Catch_Demo(void) {

   RX_Cont();

   uint8_t data[32];

   while (1) {

      uint8_t rx_stat = RX_polling(data, PCK_LEN);

      if ((rx_stat != 0)) {

         CLR_Screen();

         for (uint8_t Idx = 0; Idx < rx_stat; Idx++) {

            LPUART_Wr_Dig(data[Idx]);

         }

      }
```

```
   }

}

/*
 * Function: stby_mode
 * --------------------
 * Places rfm module into standby mode.
 */
void stby_mode(void) {

   rfm_wr(0x01, 0x81);

}

/*
 * Function: base_node
 * --------------------
 * Polls for a valid vehicle node packet and translates payload
 * data into sensor measurements. Prints measurements to terminal emulator.
 *
 */
void base_node(void) {

   CLR_Screen();

   uint8_t rx_buf[32];

   static const char *Indicators[6] = { " ",

   "RPM:      RPM",

   "Speed:      MPH",

   "Coolant Temperature:      degrees Farenheit",

   "Mass Airflow:      grams / second",

   "Intake Air Temperature:      degrees Farenheit" };

   static const uint8_t data_pos[5] = {6, 8, 22, 15, 25 );



   uint8_t Idx = 0;
```

```
for (Idx = 0; Idx < 6; Idx++) {
   Set_Pos(0, Idx);
   LPUART_Print(Indicators[Idx]);
}
RX_Cont();
while (1) {
   uint8_t pckt_sz = RX_polling(rx_buf, PCK_LEN);
   Idx = 0;
   uint8_t cnt = 0;
   if (pckt_sz != 0) {
      uint16_t Indicator_vals[10];
      char buf[32];
      while (Idx < 10) {
         Indicator_vals[cnt] = (rx_buf[Idx] << 8) | rx_buf[Idx + 1];
         Set_Pos(data_pos[cnt], cnt + 1);
         LPUART_Print("     ");
         u16_2_str(Indicator_vals[cnt], buf);
         Set_Pos(data_pos[cnt], cnt + 1);
         LPUART_Print(buf);
         Idx += 2;
         cnt++;
      }
   }
```

```
   }

}



/*
 * Function: Vehicle_node
 * --------------------
 * Turns vehicle based NUCLEO into a beacon.
 * Continuously streams CAN bus
 */
void Vehicle_node(void) {

   UART3_Init(2000000UL, 115200UL);   // USART3 init

   LPUART_ESC_Print("[2J"); // clear screen
   LPUART_ESC_Print("[H");  // move to 1,1
   LPUART_ESC_Print("[?25l"); // disable cursor on LPUART

   char line[64];

   int rpm = 0;
   int speed = 0;
   int coolant_temp = 0;
   int maf = 0;
   int intake_air = 0;

   int tx_delay = 0;

   while (1) {

      //read line from the UART
      int len = UART3_ReadLine(line, sizeof(line));

      //update CAN line here
      can_parse_line(line);

      int test_rpm = can_read_engine_rpm();
      if (test_rpm >= 0) {
         rpm = test_rpm;
      }

      int test_speed = can_read_wheel_speed();
      if (test_speed >= 0) {
         speed = test_speed;
      }
      int test_coolant_temp = can_read_coolant_temp();
      if (test_coolant_temp >= 0) {
```

```
   coolant_temp = test_coolant_temp;
}
int test_maf = can_read_maf();
if (test_maf >= 0) {
   maf = test_maf;
}
int test_intake = can_read_intake();
if (test_intake >= 0) {
   intake_air = test_intake;
}


// prepare data to send
can_build_data_stream(rpm, speed, coolant_temp, maf, intake_air);
can_build_stream_line_from_bytes();


// send data


if (tx_delay == 0) {
   rfm_tx(stream_bytes, BYTE_COUNT);
}


tx_delay++;


if (tx_delay > 5) {
   tx_delay = 0;
}


// refresh UART display, but not too quickly
LPUART_ESC_Print("[H");  // move to 1,1
LPUART_Print(line);     // print current OBD2 line
LPUART_ESC_Print("[K"); // erase from cursor to end of line
LPUART_Print("\n\r");


LPUART_Print("RPM: ");
LPUART_PrintInt(rpm);   // print rpm number
LPUART_ESC_Print("[K"); // erase from cursor to end of line
LPUART_Print("\n\r");   // next line


LPUART_Print("Speed (mph): ");
LPUART_PrintInt(speed);
LPUART_ESC_Print("[K"); // erase from cursor to end of screen
LPUART_Print("\n\r");


LPUART_Print("Coolant Temp (F): ");
LPUART_PrintInt(coolant_temp);
LPUART_ESC_Print("[K"); // erase from cursor to end of line
LPUART_Print("\n\r");


LPUART_Print("Mass Airflow Sensor (grams/s): ");
LPUART_PrintInt(maf);
LPUART_ESC_Print("[K"); // erase from cursor to end of line
```

```
        LPUART_Print("\n\r");

        LPUART_Print("Intake Air Temperature (F): ");
        LPUART_PrintInt(intake_air);
        LPUART_ESC_Print("[K"); // erase from cursor to end of line
        LPUART_Print("\n\r");

        LPUART_Print("\n\r");

        LPUART_Print("Data Stream: ");
        LPUART_Print(stream_line);
        LPUART_ESC_Print("[K");

        LPUART_Print("\n\r");

    }
}
```

LPUART.h

```
/*-----------------------------------------
 * LPUART.h
 * AUTHORS: Leo Alvelais and Derek Huie
 * CREATED: 10/20/2025
 * -----------------------------------------
 * GPIOG PINS:
 * PG7 - TX
 * PG8 - RX
 */


#ifndef INC_LPUART_H_
#define INC_LPUART_H_

//MY INCLUSIONS
#include "main.h"
#endif /* INC_LPUART_H_ */

//VARIABLES FOR EXPORT
volatile extern uint8_t up;
volatile extern uint8_t down;
volatile extern uint8_t left;
volatile extern uint8_t right;

//MY PROTOTYPES
void LPUART1_IRQHandler(void);
void LPUART1_init(void);
void LPUART_ESC_Print(char message[]);
void LPUART_Print(const char *message);
void Set_Pos(uint8_t X_Pos, uint8_t Y_pos);
void Make_Fireworks(uint8_t x_pos, uint8_t y_pos);
void Wr_to_Pos(char message[], uint8_t X_pos, uint8_t Y_pos);
void CLR_Screen(void);
void LPUART_Wr_Dig(uint16_t Dig_2_Wr);
void u16_2_str(uint16_t value, char *buf);

//MY DEFINITIONS
#define LPUART_DIV  (8889)
//#define RED_TEXT ()

//PROTOTYPES
void LPUART_Print(const char *message);
void LPUART1_init(void);
void LPUART1_IRQHandler(void);
```

LPUART.c

```
/*----------------------------------------
 * LPUART.c
 * AUTHORS: Leo Alvelais
 * CREATED: 10/20/2025
 * ----------------------------------------
 */
//INCLUSTIONS
#include <LPUART.h>
#include "main.h"
#include "delay.h"



//FUNCTIONS


/*
 * Function: LPUART_Print
 * --------------------
 * Prints a character array to the terminal emulator via LPUART
 *
 *  message: character array to be written
 */


void LPUART_Print(const char *message) {
   uint16_t iStrIdx = 0;
   while (message[iStrIdx] != 0) {
      while (!(LPUART1->ISR & USART_ISR_TXE))
         // wait for empty xmit buffer
         ;
      LPUART1->TDR = message[iStrIdx];       // send this character
      iStrIdx++;                             // advance index to next char
   }
}


/*
 * Function: LPUART_ESC_Print
 * --------------------
 * Writes escape command
 *
 *  message: escape command to be issued
 *
 */
void LPUART_ESC_Print(char message[]) {
   uint8_t iStrIdx = 0;
   //SEND ESCAPE CODE FIRST
   while (!(LPUART1->ISR & USART_ISR_TXE)) {
   }
   LPUART1->TDR = 0x1B;
```

```
   //WRITE THE REST OF THE MESSAGE CODE TO TDR
   while (message[iStrIdx] != 0) {
      while (!(LPUART1->ISR & USART_ISR_TXE))
         // wait for empty xmit buffer
         ;
      LPUART1->TDR = message[iStrIdx];       // send this character
      iStrIdx++;                             // advance index to next char
   }
}


/*
 * Function: LPUART_init
 * --------------------
 * Initializes low power UART
 * 115200 baud rate
 * 8-bit data rate.
 *
 *Configures GPIOG alternate function registers for proper UART
 *Configuration.
 *
 */


void LPUART1_init(void) {
   RCC->CCIPR &= ~(RCC_CCIPR_LPUART1SEL_Msk);
   RCC->CCIPR |= (RCC_CCIPR_LPUART1SEL_0);


   PWR->CR2 |= (PWR_CR2_IOSV);              // power avail on PG[15:2] (LPUART1)
   RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOGEN);   // enable GPIOG clock
   RCC->APB1ENR2 |= RCC_APB1ENR2_LPUART1EN; // enable LPUART clock bridge


   /* USER: configure GPIOG registers MODER/PUPDR/OTYPER/OSPEEDR then
    select AF mode and specify which function with AFR[0] and AFR[1] */


   //SET GPIOG7 AND 8 TO ALTERNATE FUNCTION MODE
   GPIOG->MODER &= ~(GPIO_MODER_MODE7 |
   GPIO_MODER_MODE8);


   GPIOG->MODER |= (GPIO_MODER_MODE7_1 |
   GPIO_MODER_MODE8_1);


   //PUSH PULL TYPE ON TX AND RX PORTS


   GPIOG->OTYPER &= ~(GPIO_OTYPER_OT7 |
   GPIO_OTYPER_OT8);


   // setting alternate function for G7 and G8
   GPIOG->AFR[0] &= ~(0x000F << GPIO_AFRL_AFSEL7_Pos); // clear PG7 nibble AF
   GPIOG->AFR[0] |= (0x0008 << GPIO_AFRL_AFSEL7_Pos); // set PG7 AF = LPUART1_TX
   GPIOG->AFR[1] &= ~(0x000F << GPIO_AFRH_AFSEL8_Pos); // clear PG8 nibble AF
   GPIOG->AFR[1] |= (0x0008 << GPIO_AFRH_AFSEL8_Pos); // set PG8 AF = LPUART1_RX
```

```
   LPUART1->CR1 &= ~(USART_CR1_M1 | USART_CR1_M0); // 8-bit data
   LPUART1->CR1 |= USART_CR1_UE;                   // enable LPUART1
   LPUART1->CR1 |= (USART_CR1_TE | USART_CR1_RE);  // enable xmit & recv
   LPUART1->CR1 |= USART_CR1_RXNEIE;        // enable LPUART1 recv interrupt
   LPUART1->ISR &= ~(USART_ISR_RXNE);       // clear Recv-Not-Empty flag
   /* USER: set baud rate register (LPUART1->BRR) */
   NVIC->ISER[2] = (1 << (LPUART1_IRQn & 0x1F));   // enable LPUART1 ISR
   //SETTING THE BAUD RATE
   LPUART1->BRR = LPUART_DIV;
   __enable_irq();                          // enable global interrupts

   //SETTING THE BAUD RATE
   LPUART1->BRR = LPUART_DIV;

}



/*
 * Function: LPUART_init
 * --------------------
 * Sets cursor to specific position in terminal emulator
 *
 * X_pos: x coordinate of cursor placement
 * Y_pos: y coordinate of cursor placement
 *
 *
 * X_pos or Y_pos can be no greater than 999
 *
 */


void Set_Pos(uint8_t X_Pos, uint8_t Y_Pos) {
   uint8_t Dig[3] = { 0, 0, 0 };
   uint8_t Dig_Idx = 0;


   Dig[0] = Y_Pos / 100;
   Dig[1] = (Y_Pos / 10) % 10;
   Dig[2] = Y_Pos % 10;


   //FOR SETTING Y POSITION
   LPUART_ESC_Print("[");
   //SEND DESIRED LINE TO IDR
   while (Dig_Idx < 3) {
      while (!(LPUART1->ISR & USART_ISR_TXE))
         ;
      LPUART1->TDR = (0x30 | Dig[Dig_Idx]);
      Dig_Idx++;
   }
   LPUART_Print(";");


   //SEND DESIRED COLUMN TO IDR
   Dig_Idx = 0;
   Dig[0] = X_Pos / 100;
```

```
   Dig[1] = (X_Pos / 10) % 10;
   Dig[2] = X_Pos % 10;

   while (Dig_Idx < 3) {
      while (!(LPUART1->ISR & USART_ISR_TXE))
         ;
      LPUART1->TDR = (0x30 | Dig[Dig_Idx]);
      Dig_Idx++;
   }
   LPUART_Print("H");
}



/*
 * Function: CLR_Screen
 * --------------------
 * Clears the terminal emulator screen and places the cursor
 * in the default position(top left corner).
 *
 */

void CLR_Screen(void) {
   LPUART_ESC_Print("[2J");
   LPUART_ESC_Print("[H");
}


/*
 * Function: LPUART_Wr_Dig
 * --------------------
 * Writes a 3 digit number to the UART terminal
 *
 * Dig_2_Wr: The digit to write to the terminal.
 *
 */
void LPUART_Wr_Dig(uint16_t Dig_2_Wr) {
   uint8_t Dig_Idx = 0;
   uint16_t Dig_Array[4] = { 0, 0, 0, 0 };
   Dig_Array[0] = (Dig_2_Wr / 1000);
   Dig_Array[1] = (Dig_2_Wr / 100) % 10;
   Dig_Array[2] = (Dig_2_Wr / 10) % 10;
   Dig_Array[3] = Dig_2_Wr % 10;

   while (Dig_Idx < 4) {

      while (!(LPUART1->ISR & USART_ISR_TXE)) {
      } // wait for empty xmit buffer
      LPUART1->TDR = (0X30 | Dig_Array[Dig_Idx]);
      Dig_Idx++;
   }
}
```

```
/*
 * Function: u16_2_str
 * --------------------
 * converts a 16 bit number to a string to print for printing numbers
 *  to the UART terminal.
 *
 * value: unsigned 16 bit number to be converted to string
 * buf: character buffer to store characters to print.
 *
 */
void u16_2_str(uint16_t value, char *buf)
{
    char tmp[6];
    int pos = 0;



    if (value == 0) {
        buf[0] = '0';
        buf[1] = '\0';
        return;
    }



    while (value > 0) {
        tmp[pos++] = (value % 10) + '0';
        value /= 10;
    }



    for (int i = 0; i < pos; i++) {
        buf[i] = tmp[pos - 1 - i];
    }

    buf[pos] = '\0';
}
```

---

delay.h

```
/*
 * delay.h
 *
 *  Created on: Oct 5, 2025
 *      Author: Leo and Anthony
 */
#include <stdint.h>

#ifndef INC_DELAY_H_
#define INC_DELAY_H_

//MY PROTOTYPES
void delay_us(uint32_t time_us);
void delay_ms(const uint32_t time_ms);
void SysTick_Init(void);

#endif /* INC_DELAY_H_ */
```

---

delay.c

```
/*
 * delay.c
 *
 *  Created on: Oct 5, 2025
 *      Author: Leo
 */


#include "delay.h"
#include "main.h"


// ------------------------------------------------- delay.c w/o #includes ---
// configure SysTick timer for use with delay_us().
// warning: breaks HAL_delay() by disabling interrupts for shorter delay timing.
void SysTick_Init(void) {
   SysTick->CTRL |= (SysTick_CTRL_ENABLE_Msk |        // enable SysTick Timer
         SysTick_CTRL_CLKSOURCE_Msk);    // select CPU clock
   SysTick->CTRL &= ~(SysTick_CTRL_TICKINT_Msk);   // disable interrupt
}


// delay in microseconds using SysTick timer to count CPU clock cycles
// do not call with 0 : error, maximum delay.
// careful calling with small nums : results in longer delays than specified:
//    e.g. @4MHz, delay_us(1) = 10-15 us delay.
void delay_us(const uint32_t time_us) {
   // set the counts for the specified delay
   SysTick->LOAD = (uint32_t) ((time_us * (SystemCoreClock / 1000000)) - 1);
   SysTick->VAL = 0;                                          // clear timer count
   SysTick->CTRL &= ~(SysTick_CTRL_COUNTFLAG_Msk);        // clear count flag
   while (!(SysTick->CTRL & SysTick_CTRL_COUNTFLAG_Msk))
      ; // wait for flag
}


void delay_ms(const uint32_t time_ms){
   delay_us(time_ms * 1000);
}
```

2. Vehicle Node

***The vehicle node has the same modules and below are the additional modules on the Vehicle node**

---

can_read.h

```
/*
 ******************************************************************************
 * EE 329 can_read.h
 ******************************************************************************
 * @file           : can_read.h
 * @brief          : can_read setup
 * project         : EE 329 Custom Project
 * authors         : Anthony di Furia
 * version         : 1
 * date            : 12/1/25
 * compiler        : STM32CubeIDE v.1.19.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics.  All rights reserved.
 ******************************************************************************
 * REVISION HISTORY
 * 12/1/25   Created file
 ******************************************************************************


 */

#ifndef INC_CAN_READ_H_
#define INC_CAN_READ_H_

#define MAX_TOKENS 10     // ID + LEN + 8 data bytes
#define TOKEN_LEN  4      // up to "7E8" + '\0'
#define BYTE_COUNT 10     // size of data stream

typedef struct {
      uint8_t hi;  // high byte
      uint8_t lo;  // low byte
} TwoBytes;

extern uint8_t stream_bytes[BYTE_COUNT];
extern char stream_line[BYTE_COUNT * 3 + 3];

#endif /* INC_CAN_READ_H_ */
```

---

can_read.c

```
/*
 ******************************************************************************
 * @file           : can_read.c
 * @brief          : contains functions that break down and process the CAN BUS
 *              line into usable values.
 * project         : EE 329 Custom Project
 * authors         : Anthony di Furia
 * version         : 1
 * date            : 12/1/25
 * compiler        : STM32CubeIDE v.1.19.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics.  All rights reserved.
 ******************************************************************************
 * REVISION HISTORY
 * 12/1/25   Created file
 ******************************************************************************

 */

#include "main.h"
#include "can_read.h"

char g_tokens[MAX_TOKENS][TOKEN_LEN];
uint16_t g_values[MAX_TOKENS];
uint8_t g_token_count = 0;

uint8_t stream_bytes[BYTE_COUNT];
char stream_line[BYTE_COUNT * 3 + 3];

/*------------------------------------------------------------------------
 * Function: hex_val
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : takes a char hex val and converts that char to the int it
 *        represents
 *
 * Inputs  : char
 * Returns : int
 *------------------------------------------------------------------------*/

static uint8_t hex_val(char c) {
   if (c >= '0' && c <= '9')
      return (uint8_t) (c - '0');
```

```
   if (c >= 'A' && c <= 'F')
      return (uint8_t) (10 + c - 'A');
   if (c >= 'a' && c <= 'f')
      return (uint8_t) (10 + c - 'a');
   return 0xFF; // invalid
}

/*-------------------------------------------------------------------------
 * Function: hex3_to_id
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : takes string id, and converts it to the int value of that id.
 * Inputs  : string
 * Returns : int
 *-------------------------------------------------------------------------*/

static uint16_t hex3_to_id(const char in[4]) {
   uint8_t a = hex_val(in[0]);
   uint8_t b = hex_val(in[1]);
   uint8_t c = hex_val(in[2]);

   if (a == 0xFF || b == 0xFF || c == 0xFF) {
      return 0;
   }

   return (uint16_t) ((a << 8) | (b << 4) | c);
}

/*-------------------------------------------------------------------------
 * Function: hex2_to_byte
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : takes byte value, and converts it to the int value of that byte.
 * Inputs  : string
 * Returns : int
 *-------------------------------------------------------------------------*/

static uint8_t hex2_to_byte(const char in[3]) {
   uint8_t hi = hex_val(in[0]);
   uint8_t lo = hex_val(in[1]);

   if (hi == 0xFF || lo == 0xFF) {
      return 0; // handle error however you like
   }

   return (uint8_t) ((hi << 4) | lo);
}

/*-------------------------------------------------------------------------
 * Function: split_hex_line
 * Author  : Anthony di Furia
```

```
 * Date    : 12/3/25
 * Purpose : takes the data string line, and splits the line by spaces into
 *        a global array of strings.
 * Inputs  : string
 * Returns : None
 *-------------------------------------------------------------------------*/

void split_hex_line(const char line[]) {
   uint8_t i = 0;  // index in line
   uint8_t t = 0;  // token index
   uint8_t pos = 0;  // index inside current token

  g_token_count = 0;

   while (line[i] != '\0') {
      char c = line[i];

      if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
         // end of current token
         if (pos > 0) {
            g_tokens[t][pos] = '\0';
            t++;
            pos = 0;

            if (t >= MAX_TOKENS) {
               break;
            }
         }
      } else {
         // part of a token
         if (pos < (TOKEN_LEN - 1)) {
            g_tokens[t][pos] = c;
            pos++;
         }
         // else: character dropped if token too long
      }

      i++;
   }

   // if we ended in middle of a token
   if (pos > 0 && t < MAX_TOKENS) {
      g_tokens[t][pos] = '\0';
      t++;
   }

   g_token_count = t;
}

/*-------------------------------------------------------------------------
 * Function: tokens_to_hex_values
```

```
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : converts each index of the global string array into its integer
 *        value, then fills the global data stream array.
 * Inputs  : None
 * Returns : None
 *-------------------------------------------------------------------------*/

void tokens_to_hex_values(void) {
   uint8_t i;

   for (i = 0; i < g_token_count; i++) {
      if (i == 0) {
         // first token is 3-digit ID
         g_values[i] = hex3_to_id(g_tokens[i]);
      } else {
         // rest are 2-digit bytes
         g_values[i] = (uint16_t) hex2_to_byte(g_tokens[i]);
      }
   }
}

/*-------------------------------------------------------------------------
 * Function: can_parse_line
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : combines the split_hex_line and tokens_to_hex_values functions
 *        to do the whole operation in one go.
 * Inputs  : CAN BUS line
 * Returns : None
 *-------------------------------------------------------------------------*/

void can_parse_line(const char line[]) {
   split_hex_line(line);
   tokens_to_hex_values();
}

/*-------------------------------------------------------------------------
 * Function: can_read_engine_rpm
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : decodes the data stream and returns the rpm as an integer
 * Inputs  : None
 * Returns : int
 *-------------------------------------------------------------------------*/

int can_read_engine_rpm(void) {
   // Need at least: ID, LEN, 04, 41, 0C, A, B
   if (g_token_count < 7) {
      return -1;
   }
```

```
   // Mode should be 0x41 (response to 0x01)
   uint8_t mode = (uint8_t) g_values[3];
   // PID should be 0x0C (engine RPM)
   uint8_t pid = (uint8_t) g_values[4];

   if (mode != 0x41 || pid != 0x0C) {
      return -1;
   }

   uint8_t A = (uint8_t) g_values[5];
   uint8_t B = (uint8_t) g_values[6];

   uint16_t raw = (uint16_t) ((A << 8) | B);  // A*256 + B
   int rpm = (int) (raw / 4);                 // OBD-II formula

   return rpm;
}

/*-------------------------------------------------------------------------
 * Function: can_read_wheel_speed
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : decodes the data stream and returns the wheel speed  in mph
 *        as an integer
 * Inputs  : None
 * Returns : int
 *-------------------------------------------------------------------------*/

// Returns wheel speed in MPH.
// Returns -1 if the current frame is not a valid wheel speed response.
int can_read_wheel_speed(void) {
   // Need at least: ID, LEN, data_len, mode, PID, A
   if (g_token_count < 6) {
      return -1;
   }

   uint8_t mode = (uint8_t) g_values[3];  // should be 0x41
   uint8_t pid = (uint8_t) g_values[4];  // should be 0x0D

   if (mode != 0x41 || pid != 0x0D) {
      return -1;
   }

   // A = speed in mph
   uint8_t A = (uint8_t) g_values[5];
   int speed_kph = (int) A;
   int speed_mph = (int) ((float) speed_kph * 0.621371f + 0.5f);

   return speed_mph;
}
```

```
/*-----------------------------------------------------------------------------
 * Function: can_read_coolant_temp
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : decodes the data stream and returns the coolant temp in fahrenheit
 *        as an int
 * Inputs  : None
 * Returns : int
 *-----------------------------------------------------------------------------*/


int can_read_coolant_temp(void) {
   // Need at least: ID, LEN, data_len, mode, PID, A
   if (g_token_count < 6) {
      return -1;
   }

   uint8_t mode = (uint8_t) g_values[3];  // should be 0x41
   uint8_t pid = (uint8_t) g_values[4];  // should be 0x05

   if (mode != 0x41 || pid != 0x05) {
      return -1;
   }

   uint8_t A = (uint8_t) g_values[5];

   // OBD-II formula: Temp (°C) = A - 40
   int temp_c = (int) A - 40;
   int temp_f = temp_c * 9 / 5 + 32;  // integer math, OK here

   return temp_f;
}

/*-----------------------------------------------------------------------------
 * Function: can_read_maf
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : decodes the data stream and returns the mass airflow amount in
 *        grams/second as an int
 * Inputs  : None
 * Returns : int
 *-----------------------------------------------------------------------------*/


int can_read_maf(void) {
   // Need at least: ID, LEN, data_len, mode, PID, A
   if (g_token_count < 7) {
      return -1;
   }

   uint8_t mode = (uint8_t) g_values[3];  // should be 0x41
   uint8_t pid = (uint8_t) g_values[4];  // should be 0x10
```

```
   if (mode != 0x41 || pid != 0x10) {
      return -1;
   }

   uint8_t A = (uint8_t) g_values[5];
   uint8_t B = (uint8_t) g_values[6];

   int raw = ((256 * A) + B);
   int maf_gps = (int) (raw / 100);

   return maf_gps;
}

/*-----------------------------------------------------------------------------
 * Function: can_read_intake
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : decodes the data stream and returns the intake air temp in
 *        fahrenheit as an int
 * Inputs  : None
 * Returns : int
 *---------------------------------------------------------------------------*/

int can_read_intake(void) {
   // Need at least: ID, LEN, data_len, mode, PID, A
   if (g_token_count < 6) {
      return -1;
   }

   uint8_t mode = (uint8_t) g_values[3];  // should be 0x41
   uint8_t pid = (uint8_t) g_values[4];  // should be 0x0F

   if (mode != 0x41 || pid != 0x0F) {
      return -1;
   }

   uint8_t A = (uint8_t) g_values[5];

   int16_t iat_C = (int16_t) A - 40;
   int iat_f = iat_C * 9 / 5 + 32;  // integer math, OK here

   return iat_f;
}

/*-----------------------------------------------------------------------------
 * Function: int_to_two_bytes
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : takes an integer, and returns a typedef struct "TwoBytes" that
 *        contains a high and low byte, as each value is two bytes.
 * Inputs  : integer
```

```
 * Returns : TwoBytes
 *-------------------------------------------------------------------------*/

TwoBytes int_to_two_bytes(int value) {
   TwoBytes out;

   if (value < 0) {
      value = 0;
   } else if (value > 65535) {
      value = 65535;
   }

   uint16_t v = (uint16_t) value;

   out.hi = (uint8_t) (v >> 8);   // upper 8 bits
   out.lo = (uint8_t) (v & 0xFF); // lower 8 bits

   return out;
}

/*-------------------------------------------------------------------------
 * Function: can_build_data_stream
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : takes all of the values received from the car and encodes them
 *        into a global of stream_bytes
 * Inputs  : int
 * Returns : None
 *-------------------------------------------------------------------------*/

void can_build_data_stream(int rpm, int speed, int coolant, int maf, int intake) {
   TwoBytes r = int_to_two_bytes(rpm);
   TwoBytes sp = int_to_two_bytes(speed);
   TwoBytes ct = int_to_two_bytes(coolant);
   TwoBytes mf = int_to_two_bytes(maf);
   TwoBytes in = int_to_two_bytes(intake);

   stream_bytes[0] = r.hi;
   stream_bytes[1] = r.lo;

   stream_bytes[2] = sp.hi;
   stream_bytes[3] = sp.lo;

   stream_bytes[4] = ct.hi;
   stream_bytes[5] = ct.lo;

   stream_bytes[6] = mf.hi;
   stream_bytes[7] = mf.lo;

   stream_bytes[8] = in.hi;
```

```
   stream_bytes[9] = in.lo;
}


/*-------------------------------------------------------------------------
 * Function: can_build_stream_line_from_bytes
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : For debug purposes: prints a string representation of the data
 *        stream.
 * Inputs  : None
 * Returns : None
 *-------------------------------------------------------------------------*/


// to print the string of bytes
void can_build_stream_line_from_bytes(void) {
   int pos = 0;


   for (int i = 0; i < BYTE_COUNT; i++) {
      uint8_t b = stream_bytes[i];
      uint8_t hi = (b >> 4) & 0x0F;
      uint8_t lo = b & 0x0F;


      stream_line[pos++] = (hi < 10) ? ('0' + hi) : ('A' + (hi - 10));
      stream_line[pos++] = (lo < 10) ? ('0' + lo) : ('A' + (lo - 10));
      stream_line[pos++] = ' ';
   }


   stream_line[pos++] = '\r';
   stream_line[pos++] = '\n';
   stream_line[pos] = '\0';
}
```

---

uart3.h

```
/*
*******************************************************************************
* EE 329 uart3.h
*******************************************************************************
* @file           : uart3.h
* @brief          : uart3 setup
* project         : EE 329 Custom Project
* authors         : Anthony di Furia
* version         : 1
* date            : 12/1/25
* compiler        : STM32CubeIDE v.1.19.0
* target          : NUCLEO-L4A6ZG
* clocks          : 4 MHz MSI to AHB2
* @attention      : (c) 2025 STMicroelectronics.  All rights reserved.
*******************************************************************************
* REVISION HISTORY
* 12/1/25    Created file
*******************************************************************************

 */



#ifndef UART3_H
#define UART3_H

#include <stdint.h>
#include "stm32l4xx.h"   // CMSIS device header, no HAL required

// Initialize USART3 for RX (and optional TX) on PD9 (and PD8) at given baud.
// pclk1_hz must be the APB1 clock frequency (in Hz),
// e.g. 80000000 for 80 MHz.
void UART3_Init(uint32_t pclk1_hz, uint32_t baudrate);

// Blocking read of a single character from USART3.
// Returns the received byte (0-255).
int UART3_ReadChar(void);

// Blocking read of one text line from USART3.
// - buffer: where to store the line (null-terminated)
// - maxLen: size of buffer in bytes
// Returns number of chars stored (not counting '\0').
int UART3_ReadLine(char *buffer, uint16_t maxLen);

#endif // UART3_H
```

---

uart3.c

```
/*
 ******************************************************************************
 * @file           : uart3.c
 * @brief          : contains the code to enable use of the UART3 on the STM32.
 * project         : EE 329 Custom Project
 * authors         : Anthony di Furia
 * version         : 1
 * date            : 12/1/25
 * compiler        : STM32CubeIDE v.1.19.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics.  All rights reserved.
 ******************************************************************************
 * UART3 COMMUNICATION PLAN :
 * Configure USART3 on PD9 (RX) for receiving serial data from Arduino
 * 8 data bits, no parity, 1 stop bit, oversampling by 16
 * Receiver enabled, optional transmitter on PD8
 * Blocking read functions with automatic error handling
 ******************************************************************************
 * UART3 WIRING (pinout NUCLEO-L4A6ZG)
 * Arduino                                                L4A6ZG
 *
 *PD1 TX -- 10k resistor --------- node ------------PD9 (USART3_RX / D0)
 *                                  |
 *                               20k resistor
 *                                  |
 *                                GND
 * Common GND between Arduino and Nucleo required
 * Voltage divider: 5V Arduino signal -> 3.3V STM32 logic level
 * Optional: PD8 (USART3_TX / D1) for transmit capability
 ******************************************************************************
 * SPI between Arduino Uno and Can bus shield
 *
 * Shield                                  Arduino
 * JP5 - 5V  ---------------------------- POWER - 5
 * JP5 - 3V3 ---------------------------- POWER - 4
 * JP5 - GND ---------------------------- POWER - 6
 * JP1 - 13  ---------------------------- IOH - 6
 * JP1 - 12  ---------------------------- IOH - 5
 * JP1 - 11  ---------------------------- IOH - 4
 * JP1 - 9   ---------------------------- IOH - 2
 ******************************************************************************
 * REVISION HISTORY
 * 12/1/25   Created file
 ******************************************************************************
 */

#include "UART3.h"
```

```c
#include <stdio.h>      // for snprintf in LPUART_PrintInt

/*-------------------------------------------------------------------------
 * Function: UART3_init
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : initialize the UART3, given a certain clock speed and baud rate.
 * Inputs  : clock speed, baud rate
 * Returns : None
 *-------------------------------------------------------------------------*/
void UART3_Init(uint32_t pclk1_hz, uint32_t baudrate) {
   // 1) Enable clocks for GPIOD and USART3

   // Enable GPIOD clock on AHB2
   RCC->AHB2ENR |= RCC_AHB2ENR_GPIODEN;

   // Enable USART3 clock on APB1
   RCC->APB1ENR1 |= RCC_APB1ENR1_USART3EN;   // USART3EN bit

   // 2) Configure PD9 as AF7 (USART3_RX)
   // PD9: MODER9[1:0] = 10 (alternate function)
   GPIOD->MODER &= ~(0x3U << (9U * 2U));
   GPIOD->MODER |= (0x2U << (9U * 2U));

   // Output type: push-pull
   GPIOD->OTYPER &= ~(1U << 9);

   // Speed: high speed
   GPIOD->OSPEEDR &= ~(0x3U << (9U * 2U));
   GPIOD->OSPEEDR |= (0x3U << (9U * 2U));

   // Pull-up on RX (01 = pull-up)
   GPIOD->PUPDR &= ~(0x3U << (9U * 2U));
   GPIOD->PUPDR |= (0x1U << (9U * 2U));

   // Alternate function AF7 on PD9:
   // AFRH index for pin 9 is (9-8)*4 = 4
   GPIOD->AFR[1] &= ~(0xFU << ((9U - 8U) * 4U));
   GPIOD->AFR[1] |= (0x7U << ((9U - 8U) * 4U));  // AF7: USART3_RX

   // 3) Configure USART3

   // Disable before config
   USART3->CR1 &= ~USART_CR1_UE;

   // 8 data bits, no parity, oversampling by 16
   USART3->CR1 = 0;
   USART3->CR2 = 0;
   USART3->CR3 = 0;
```

```
   // 4) Baud rate: BRR = PCLK1 / baudrate
   uint32_t usartdiv = (pclk1_hz + (baudrate / 2U)) / baudrate;
   USART3->BRR = usartdiv;

   // Enable receiver (and TX if needed)
   USART3->CR1 |= USART_CR1_RE;
   // USART3->CR1 |= USART_CR1_TE;  // enable if you wire TX

   // Enable USART3
   USART3->CR1 |= USART_CR1_UE;

}

/*-------------------------------------------------------------------------
 * Function: UART3_ReadChar
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : reads character from the UART3 Rx pin
 * Inputs  : None
 * Returns : integer
 *-------------------------------------------------------------------------*/

int UART3_ReadChar(void) {
   while (1) {
      uint32_t isr = USART3->ISR;

      // Any error? (Overrun / Framing / Noise)
      if (isr & (USART_ISR_ORE | USART_ISR_FE | USART_ISR_NE)) {
         // Read RDR to clear RXNE / data
         volatile uint32_t tmp = USART3->RDR;
         (void) tmp;

         // Clear the error flags in ICR
         USART3->ICR = USART_ICR_ORECF | USART_ICR_FECF | USART_ICR_NCF;
         // then continue the loop and wait for a clean byte
         continue;
      }

      // Normal receive: RXNE set
      if (isr & USART_ISR_RXNE) {
         return (int) (USART3->RDR & 0xFF);
      }

      // else: no data yet, spin
   }
}
```

```
/*-------------------------------------------------------------------------
 * Function: UART3_ReadLine
 * Author  : Anthony di Furia
 * Date    : 12/3/25
 * Purpose : reads entire line received via the UART3 Rx pin
 * Inputs  : None
 * Returns : integer
 *-------------------------------------------------------------------------*/


int UART3_ReadLine(char *buffer, uint16_t maxLen) {
   if (buffer == NULL || maxLen < 2) {
      return 0;
   }


   uint16_t idx = 0;


   while (1) {
      int c = UART3_ReadChar();   // blocking


      if (c == '\n') {
         // Strip trailing '\r' if present (Arduino sends "\r\n")
         if (idx > 0 && buffer[idx - 1] == '\r') {
            idx--;
         }
         buffer[idx] = '\0';
         return (int) idx;
      }


      if (idx < (maxLen - 1)) {
         buffer[idx++] = (char) c;
      } else {
         // Buffer full, terminate and return what's there
         buffer[maxLen - 1] = '\0';
         return (int) (maxLen - 1);
      }
   }
}
```

3. Vehicle Node ARDUNIO Code

OBD-II_PIDs.ino

```
/*********************************************************************************************
    OBD-II_PIDs SPAM LOGGER
    Based on LOOVEE @ JUN24, 2017

    - Periodically sends PID queries over OBD-II
    - Prints each received CAN frame as:
        0x<ID> <LEN> <DATA...>

*********************************************************************************************/
#include <SPI.h>

#define CAN_2515
// #define CAN_2518FD

// Set SPI CS Pin according to your hardware

#if defined(SEEED_WIO_TERMINAL) && defined(CAN_2518FD)
// For Wio Terminal w/ MCP2518FD RPi Hat:
// Channel 0 SPI_CS Pin: BCM 8
// Channel 1 SPI_CS Pin: BCM 7
// Interupt Pin: BCM25
const int SPI_CS_PIN  = BCM8;
const int CAN_INT_PIN = BCM25;
#else

// For Arduino MCP2515 Hat:
// the cs pin of the version after v1.1 is default to D9
// v0.9b and v1.0 is default D10
const int SPI_CS_PIN = 9;
const int CAN_INT_PIN = 2;
#endif



#ifdef CAN_2518FD
#include "mcp2518fd_can.h"
mcp2518fd CAN(SPI_CS_PIN); // Set CS pin
#endif

#ifdef CAN_2515
#include "mcp2515_can.h"
mcp2515_can CAN(SPI_CS_PIN); // Set CS pin
#endif

#define PID_ENGIN_PRM       0x0C
```

```
#define PID_VEHICLE_SPEED   0x0D
#define PID_COOLANT_TEMP    0x05

// ==== NEW: extra PIDs ====
#define PID_INTAKE_AIR_TEMP 0x0F   // Intake Air Temperature
#define PID_MAF             0x10   // Mass Air Flow
// =========================

#define CAN_ID_PID          0x7DF

unsigned char PID_INPUT;
unsigned char getPid    = 0;

// ==== NEW: periodic query settings ====
unsigned long lastQueryTime = 0;
const unsigned long QUERY_INTERVAL_MS = 100;  // how often to send (ms)

// list of PIDs to cycle through
const uint8_t pidList[] = {
    PID_ENGIN_PRM,
    PID_VEHICLE_SPEED,
    PID_COOLANT_TEMP,
    PID_MAF,             // NEW
    PID_INTAKE_AIR_TEMP  // NEW
};
const uint8_t pidListCount = sizeof(pidList) / sizeof(pidList[0]);
uint8_t pidIndex = 0;
// =====================================

void set_mask_filt() {
    /*
        set mask, set both the mask to 0x7FC
        (only match 0x7E8-0x7EF range if you want; here all set to 0x7E8 just like original)
    */
    CAN.init_Mask(0, 0, 0x7FC);
    CAN.init_Mask(1, 0, 0x7FC);

    /*
        set filter, we can receive id 0x7E8
        you can change these to other ECU response IDs if needed
    */
    CAN.init_Filt(0, 0, 0x7E8);
    CAN.init_Filt(1, 0, 0x7E8);

    CAN.init_Filt(2, 0, 0x7E8);
    CAN.init_Filt(3, 0, 0x7E8);
    CAN.init_Filt(4, 0, 0x7E8);
    CAN.init_Filt(5, 0, 0x7E8);
}

void sendPid(unsigned char __pid) {
```

```
    unsigned char tmp[8] = {0x02, 0x01, __pid, 0, 0, 0, 0, 0};
    CAN.sendMsgBuf(CAN_ID_PID, 0, 8, tmp);
}


// helper to print one CAN frame as:
// 0x7E3 03 41 0C 1A F8 00 00
void printCanFrame(unsigned long id, unsigned char len, unsigned char *buf) {
    SERIAL_PORT_MONITOR.print("0x");
    SERIAL_PORT_MONITOR.print(id, HEX);
    SERIAL_PORT_MONITOR.print(" ");


    // print length as 2-digit hex (optional)
    if (len < 0x10) SERIAL_PORT_MONITOR.print("0");
    SERIAL_PORT_MONITOR.print(len, HEX);


    SERIAL_PORT_MONITOR.print(" ");


    for (int i = 0; i < len; i++) {
        if (buf[i] < 0x10) SERIAL_PORT_MONITOR.print("0");  // leading zero
        SERIAL_PORT_MONITOR.print(buf[i], HEX);
        if (i < len - 1) SERIAL_PORT_MONITOR.print(" ");
    }
    SERIAL_PORT_MONITOR.println();
}


void setup() {
    SERIAL_PORT_MONITOR.begin(115200);
    while (!Serial) {}


    while (CAN_OK != CAN.begin(CAN_500KBPS)) {              // init can bus : baudrate = 500k
        SERIAL_PORT_MONITOR.println("CAN init fail, retry...");
        delay(100);
    }
    SERIAL_PORT_MONITOR.println("CAN init ok!");
    set_mask_filt();
}


void loop() {
    // 1) periodically send PID queries ("spam" the bus)
    unsigned long now = millis();
    if (now - lastQueryTime >= QUERY_INTERVAL_MS) {
        lastQueryTime = now;


        // cycle through predefined PIDs
        sendPid(pidList[pidIndex]);
        pidIndex++;
        if (pidIndex >= pidListCount) {
            pidIndex = 0;
        }
    }
```

```
    // 2) process incoming frames and print them in compact format
    taskCanRecv();

    // 3) still allow manual PID entry over serial (optional)
    taskDbg();
}

void taskCanRecv() {
    unsigned char len = 0;
    unsigned char buf[8];

    if (CAN_MSGAVAIL == CAN.checkReceive()) {  // check if we got data
        CAN.readMsgBuf(&len, buf);             // read data
        unsigned long rxId = CAN.getCanId();

        // print in one-line format
        printCanFrame(rxId, len, buf);
    }
}

void taskDbg() {
    while (SERIAL_PORT_MONITOR.available()) {
        char c = SERIAL_PORT_MONITOR.read();

        if (c >= '0' && c <= '9') {
            PID_INPUT *= 0x10;
            PID_INPUT += c - '0';

        } else if (c >= 'A' && c <= 'F') {
            PID_INPUT *= 0x10;
            PID_INPUT += 10 + c - 'A';
        } else if (c >= 'a' && c <= 'f') {
            PID_INPUT *= 0x10;
            PID_INPUT += 10 + c - 'a';
        } else if (c == '\n') { // END
            getPid = 1;
        }
    }

    if (getPid) {       // GET A PID from serial
        getPid = 0;
        sendPid(PID_INPUT);
        PID_INPUT = 0;
    }
}
// END FILE
```

SPI.c

```
/*
 * Copyright (c) 2010 by Cristian Maglie <c.maglie@arduino.cc>
 * Copyright (c) 2014 by Paul Stoffregen <paul@pjrc.com> (Transaction API)
 * Copyright (c) 2014 by Matthijs Kooijman <matthijs@stdin.nl> (SPISettings AVR)
 * Copyright (c) 2014 by Andrew J. Kroll <xxxajk@gmail.com> (atomicity fixes)
 * SPI Master library for arduino.
 *
 * This file is free software; you can redistribute it and/or modify
 * it under the terms of either the GNU General Public License version 2
 * or the GNU Lesser General Public License version 2.1, both as
 * published by the Free Software Foundation.
 */


#ifndef _SPI_H_INCLUDED
#define _SPI_H_INCLUDED


#include <Arduino.h>


// SPI_HAS_TRANSACTION means SPI has beginTransaction(), endTransaction(),
// usingInterrupt(), and SPISetting(clock, bitOrder, dataMode)
#define SPI_HAS_TRANSACTION 1


// SPI_HAS_NOTUSINGINTERRUPT means that SPI has notUsingInterrupt() method
#define SPI_HAS_NOTUSINGINTERRUPT 1


// SPI_ATOMIC_VERSION means that SPI has atomicity fixes and what version.
// This way when there is a bug fix you can check this define to alert users
// of your code if it uses better version of this library.
// This also implies everything that SPI_HAS_TRANSACTION as documented above is
// available too.
#define SPI_ATOMIC_VERSION 1


// Uncomment this line to add detection of mismatched begin/end transactions.
// A mismatch occurs if other libraries fail to use SPI.endTransaction() for
// each SPI.beginTransaction().  Connect an LED to this pin.  The LED will turn
// on if any mismatch is ever detected.
//#define SPI_TRANSACTION_MISMATCH_LED 5


#ifndef LSBFIRST
#define LSBFIRST 0
#endif
#ifndef MSBFIRST
#define MSBFIRST 1
#endif


#define SPI_CLOCK_DIV4 0x00
```

```
#define SPI_CLOCK_DIV16 0x01
#define SPI_CLOCK_DIV64 0x02
#define SPI_CLOCK_DIV128 0x03
#define SPI_CLOCK_DIV2 0x04
#define SPI_CLOCK_DIV8 0x05
#define SPI_CLOCK_DIV32 0x06

#define SPI_MODE0 0x00
#define SPI_MODE1 0x04
#define SPI_MODE2 0x08
#define SPI_MODE3 0x0C

#define SPI_MODE_MASK 0x0C  // CPOL = bit 3, CPHA = bit 2 on SPCR
#define SPI_CLOCK_MASK 0x03  // SPR1 = bit 1, SPR0 = bit 0 on SPCR
#define SPI_2XCLOCK_MASK 0x01  // SPI2X = bit 0 on SPSR

// define SPI_AVR_EIMSK for AVR boards with external interrupt pins
#if defined(EIMSK)
  #define SPI_AVR_EIMSK  EIMSK
#elif defined(GICR)
  #define SPI_AVR_EIMSK  GICR
#elif defined(GIMSK)
  #define SPI_AVR_EIMSK  GIMSK
#endif

class SPISettings {
public:
  SPISettings(uint32_t clock, uint8_t bitOrder, uint8_t dataMode) {
    if (__builtin_constant_p(clock)) {
      init_AlwaysInline(clock, bitOrder, dataMode);
    } else {
      init_MightInline(clock, bitOrder, dataMode);
    }
  }
  SPISettings() {
    init_AlwaysInline(4000000, MSBFIRST, SPI_MODE0);
  }
private:
  void init_MightInline(uint32_t clock, uint8_t bitOrder, uint8_t dataMode) {
    init_AlwaysInline(clock, bitOrder, dataMode);
  }
  void init_AlwaysInline(uint32_t clock, uint8_t bitOrder, uint8_t dataMode)
    __attribute__((__always_inline__)) {
    // Clock settings are defined as follows. Note that this shows SPI2X
    // inverted, so the bits form increasing numbers. Also note that
    // fosc/64 appears twice
    // SPR1 SPR0 ~SPI2X Freq
    //   0    0     0   fosc/2
    //   0    0     1   fosc/4
    //   0    1     0   fosc/8
    //   0    1     1   fosc/16
    //   1    0     0   fosc/32
    //   1    0     1   fosc/64
    //   1    1     0   fosc/64
```

```
  //   1    1     1   fosc/128

  // We find the fastest clock that is less than or equal to the
  // given clock rate. The clock divider that results in clock_setting
  // is 2 ^^ (clock_div + 1). If nothing is slow enough, we'll use the
  // slowest (128 == 2 ^^ 7, so clock_div = 6).
  uint8_t clockDiv;

  // When the clock is known at compile time, use this if-then-else
  // cascade, which the compiler knows how to completely optimize
  // away. When clock is not known, use a loop instead, which generates
  // shorter code.
  if (__builtin_constant_p(clock)) {
    if (clock >= F_CPU / 2) {
      clockDiv = 0;
    } else if (clock >= F_CPU / 4) {
      clockDiv = 1;
    } else if (clock >= F_CPU / 8) {
      clockDiv = 2;
    } else if (clock >= F_CPU / 16) {
      clockDiv = 3;
    } else if (clock >= F_CPU / 32) {
      clockDiv = 4;
    } else if (clock >= F_CPU / 64) {
      clockDiv = 5;
    } else {
      clockDiv = 6;
    }
  } else {
    uint32_t clockSetting = F_CPU / 2;
    clockDiv = 0;
    while (clockDiv < 6 && clock < clockSetting) {
      clockSetting /= 2;
      clockDiv++;
    }
  }

  // Compensate for the duplicate fosc/64
  if (clockDiv == 6)
  clockDiv = 7;

  // Invert the SPI2X bit
  clockDiv ^= 0x1;

  // Pack into the SPISettings class
  spcr = _BV(SPE) | _BV(MSTR) | ((bitOrder == LSBFIRST) ? _BV(DORD) : 0) |
    (dataMode & SPI_MODE_MASK) | ((clockDiv >> 1) & SPI_CLOCK_MASK);
  spsr = clockDiv & SPI_2XCLOCK_MASK;
 }
 uint8_t spcr;
 uint8_t spsr;
 friend class SPIClass;
};
```

```
class SPIClass {
public:
  // Initialize the SPI library
  static void begin();

  // If SPI is used from within an interrupt, this function registers
  // that interrupt with the SPI library, so beginTransaction() can
  // prevent conflicts.  The input interruptNumber is the number used
  // with attachInterrupt.  If SPI is used from a different interrupt
  // (eg, a timer), interruptNumber should be 255.
  static void usingInterrupt(uint8_t interruptNumber);
  // And this does the opposite.
  static void notUsingInterrupt(uint8_t interruptNumber);
  //       the usingInterrupt and notUsingInterrupt functions should
  // not to be called from ISR context or inside a transaction.
  // For details see:
  // https://github.com/arduino/Arduino/pull/2381
  // https://github.com/arduino/Arduino/pull/2449

  // Before using SPI.transfer() or asserting chip select pins,
  // this function is used to gain exclusive access to the SPI bus
  // and configure the correct settings.
  inline static void beginTransaction(SPISettings settings) {
    if (interruptMode > 0) {
      uint8_t sreg = SREG;
      noInterrupts();

      #ifdef SPI_AVR_EIMSK
      if (interruptMode == 1) {
        interruptSave = SPI_AVR_EIMSK;
        SPI_AVR_EIMSK &= ~interruptMask;
        SREG = sreg;
      } else
      #endif
      {
        interruptSave = sreg;
      }
    }

    #ifdef SPI_TRANSACTION_MISMATCH_LED
    if (inTransactionFlag) {
      pinMode(SPI_TRANSACTION_MISMATCH_LED, OUTPUT);
      digitalWrite(SPI_TRANSACTION_MISMATCH_LED, HIGH);
    }
    inTransactionFlag = 1;
    #endif

    SPCR = settings.spcr;
    SPSR = settings.spsr;
  }
```

```
// Write to the SPI bus (MOSI pin) and also receive (MISO pin)
inline static uint8_t transfer(uint8_t data) {
  SPDR = data;
  /*
   * The following NOP introduces a small delay that can prevent the wait
   * loop form iterating when running at the maximum speed. This gives
   * about 10% more speed, even if it seems counter-intuitive. At lower
   * speeds it is unnoticed.
   */
  asm volatile("nop");
  while (!(SPSR & _BV(SPIF))) ; // wait
  return SPDR;
}
inline static uint16_t transfer16(uint16_t data) {
  union { uint16_t val; struct { uint8_t lsb; uint8_t msb; }; } in, out;
  in.val = data;
  if (!(SPCR & _BV(DORD))) {
    SPDR = in.msb;
    asm volatile("nop"); // See transfer(uint8_t) function
    while (!(SPSR & _BV(SPIF))) ;
    out.msb = SPDR;
    SPDR = in.lsb;
    asm volatile("nop");
    while (!(SPSR & _BV(SPIF))) ;
    out.lsb = SPDR;
  } else {
    SPDR = in.lsb;
    asm volatile("nop");
    while (!(SPSR & _BV(SPIF))) ;
    out.lsb = SPDR;
    SPDR = in.msb;
    asm volatile("nop");
    while (!(SPSR & _BV(SPIF))) ;
    out.msb = SPDR;
  }
  return out.val;
}
inline static void transfer(void *buf, size_t count) {
  if (count == 0) return;
  uint8_t *p = (uint8_t *)buf;
  SPDR = *p;
  while (--count > 0) {
    uint8_t out = *(p + 1);
    while (!(SPSR & _BV(SPIF))) ;
    uint8_t in = SPDR;
    SPDR = out;
    *p++ = in;
  }
  while (!(SPSR & _BV(SPIF))) ;
  *p = SPDR;
}
// After performing a group of transfers and releasing the chip select
// signal, this function allows others to access the SPI bus
inline static void endTransaction(void) {
```

```
    #ifdef SPI_TRANSACTION_MISMATCH_LED
    if (!inTransactionFlag) {
      pinMode(SPI_TRANSACTION_MISMATCH_LED, OUTPUT);
      digitalWrite(SPI_TRANSACTION_MISMATCH_LED, HIGH);
    }
    inTransactionFlag = 0;
    #endif

    if (interruptMode > 0) {
      #ifdef SPI_AVR_EIMSK
      uint8_t sreg = SREG;
      #endif
      noInterrupts();
      #ifdef SPI_AVR_EIMSK
      if (interruptMode == 1) {
        SPI_AVR_EIMSK = interruptSave;
        SREG = sreg;
      } else
      #endif
      {
        SREG = interruptSave;
      }
    }
  }

  // Disable the SPI bus
  static void end();

  // This function is deprecated.  New applications should use
  // beginTransaction() to configure SPI settings.
  inline static void setBitOrder(uint8_t bitOrder) {
    if (bitOrder == LSBFIRST) SPCR |= _BV(DORD);
    else SPCR &= ~(_BV(DORD));
  }
  // This function is deprecated.  New applications should use
  // beginTransaction() to configure SPI settings.
  inline static void setDataMode(uint8_t dataMode) {
    SPCR = (SPCR & ~SPI_MODE_MASK) | dataMode;
  }
  // This function is deprecated.  New applications should use
  // beginTransaction() to configure SPI settings.
  inline static void setClockDivider(uint8_t clockDiv) {
    SPCR = (SPCR & ~SPI_CLOCK_MASK) | (clockDiv & SPI_CLOCK_MASK);
    SPSR = (SPSR & ~SPI_2XCLOCK_MASK) | ((clockDiv >> 2) & SPI_2XCLOCK_MASK);
  }
  // These undocumented functions should not be used.  SPI.transfer()
  // polls the hardware flag which is automatically cleared as the
  // AVR responds to SPI's interrupt
  inline static void attachInterrupt() { SPCR |= _BV(SPIE); }
  inline static void detachInterrupt() { SPCR &= ~_BV(SPIE); }

private:
  static uint8_t initialized;
  static uint8_t interruptMode; // 0=none, 1=mask, 2=global
```

```
  static uint8_t interruptMask; // which interrupts to mask
  static uint8_t interruptSave; // temp storage, to restore state
  #ifdef SPI_TRANSACTION_MISMATCH_LED
  static uint8_t inTransactionFlag;
  #endif
};

extern SPIClass SPI;

#endif
```

D. Other Flow Charts

Arduino Flow Chart:

